Rockchip平台PCIe设备虚拟化开发指南

文件标识：RK-KF-YF-170

发布版本：V2.1.1

日期：2026-06-05

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

该方案基于QEMU + ARM KVM + PCIe + SMMU + VFIO来实现外设的虚拟化，主要目的是实现在虚拟机中直接访问PCIe外设，依据本指南可以实现在虚拟机中挂载PCIe NVME SSD以及在虚拟机中实现PCIe网卡的连接。

产品版本

芯片名称	内核版本
带SMMU的Rockchip芯片	4.19
RK3588	6.12

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

日期	版本	作者	修改说明
2021-03-30	V1.0.0	薛小明	初始版本
2025-09-28	V2.0.0	薛小明	添加RK3588支持
2025-10-23	V2.1.0	薛小明	添加支持SR-IOV功能
2026-06-05	V2.1.1	薛小明	修正格式

目录

Rockchip平台PCIe设备虚拟化开发指南

1. 环境搭建
    - 1.1 Guest OS内核配置
    - 1.2 Guest OS ramdisk准备
    - 1.3 HOST OS内核配置
    - 1.4 QEMU准备
    - 1.5 Host OS ramdisk准备
2. DTS 配置
    - 2.1 SMMU配置
    - 2.2 PCIe补充配置
3. 运行
4. 遗留问题
5. 修改dtb

1. 环境搭建

1.1 Guest OS内核配置

准备内核

```
git clone https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux-stable.git
```

使用defconfig生成初始.config

```
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- defconfig
```

继续打开如下配置。

```
CONFIG_ARM64_VA_BITS_39=y
CONFIG_ARM64_VA_BITS=39
CONFIG_R8169=y
CONFIG_NVME_CORE=y
CONFIG_BLK_DEV_NVME=y
CONFIG_NVME_HWMON=y
```

关闭如下配置。

```
CONFIG_HUGETLBFS
CONFIG_HUGETLB_PAGE
```

使用以下命令生成guest os内核固件。

```
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -j8
```

固件位置。

```
arch/arm64/boot/Image
```

1.2 Guest OS ramdisk准备

包含busybox工具，lspci，ifconfig工具。

1.3 HOST OS内核配置

使用RK3588 linux配置生成初始.config:

```
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 rockchip_linux_defconfig
rk3588_linux.config
```

再使能如下配置。

```
CONFIG_KVM=y
CONFIG_VFIO=y
CONFIG_VFIO_PCI=y
CONFIG_IOMMU_SUPPORT=y
CONFIG_ARM_SMMU_V3=y
CONFIG_VFIO_IOMMU_TYPE1=y
CONFIG_BLK_DEV_NVME=y
CONFIG_R8169=y
CONFIG_PCIE_DW=y
CONFIG_PCIE_DW_HOST=y
CONFIG_PCIE_DW_ROCKCHIP=y
CONFIG_VIRTUALIZATION=y
CONFIG_KVM_VFIO=y
CONFIG_PCI_IOV=y
```

1.4 QEMU准备

使用Rockchip的buildroot工程编译qemu，可执行文件名称qemu-system-aarch64。

1.5 Host OS ramdisk准备

使用Rockchip的buildroot工程生成ramdisk，包含qemu-system-aarch64，guest os内核固件，guest os ramdisk固件。

Rockchip的buildroot工程配置以及编译方法参考buildroot使用文档，本指南提供预先编译好的ramdisk目录，包含需要的所有固件，

在ubuntu环境下，使用abootimg-pack-initrd，会在ramdisk同级目录生成initrd.img固件，使用abootimg-unpack-initrd解压initrd.img。

为了方便测试，可以直接将initramfs编译进内核

```
CONFIG_INITRAMFS_SOURCE="/home/xxm/initramfs"
```

2. DTS 配置

2.1 SMMU配置

```
compatible = "arm,smmu-v3";
```

必须配置，用于匹配ARM SMMU-v3驱动。

以下寄存器以及中断配置以RK3588为例。

```
reg = <0x0 0xfc900000 0x0 0x100000>;
```

ARM SMMU-v3基地址以及大小。

```
interrupts = <GIC_SPI 369 IRQ_TYPE_EDGE_RISING>,
             <GIC_SPI 371 IRQ_TYPE_EDGE_RISING>,
             <GIC_SPI 374 IRQ_TYPE_EDGE_RISING>,
             <GIC_SPI 367 IRQ_TYPE_EDGE_RISING>;
interrupt-names = "eventq", "gerror", "priq", "cmdq-sync";
```

详见Documentation/devicetree/bindings/iommu/arm,smmu-v3.txt。

```
#iommu-cells = <1>;
```

必须为1，用于描述设备的stream ID，详见Documentation/devicetree/bindings/iommu/arm,smmu-v3.txt。

2.2 PCIe补充配置

```
&mmu600_pcie {
    status = "okay";
};
```

```
&pcie2x1l1 {
    iommu-map = <0x0 &mmu600_pcie 0x0 0x10000>;
};
&pcie3x4 {
    iommu-map = <0x0 &mmu600_pcie 0x0 0x10000>;
};
```

为PCIe外设创建SMMU页表。

3. 运行

1. 卸载宿主机PCIe设备驱动

   使用lspci得到设备地址，假设设备地址为

   ```
   0000:00:00.0 Class 0604: Device 1d87:3588 (rev 01)
   0000:01:00.0 Class 0108: Device 1e0f:0008 (rev 01)
   0003:30:00.0 Class 0604: Device 1d87:3588 (rev 01)
   0003:31:00.0 Class 0200: Device 10ec:8168 (rev 15)
   ```

   执行以下命令卸载。

   ```
   echo 0000:01:00.0 > /sys/bus/pci/devices/0000:01:00.0/driver/unbind
   echo 0003:31:00.0 > /sys/bus/pci/devices/0003:31:00.0/driver/unbind
   ```

2. 将PCIe设备重新挂载到VFIO-PCI驱动

   使用lspci得到设备vendor id和device id，以Realtek 8169 PCIe网卡和PCIe NVME SSD为例，vendor id和device id分别为10ec:8168和1e0f:0008，使用以下命令挂载。

```
echo 10ec 8168 > /sys/bus/pci/drivers/vfio-pci/new_id
echo 1e0f 0008 > /sys/bus/pci/drivers/vfio-pci/new_id
```

3. 运行qemu

```
qemu-system-aarch64 -nographic -monitor none -m 128 -machine
virt,highmem=off -serial stdio -cpu host -enable-kvm -net none -device vfio-
pci,host=0000:01:00.0 -device vfio-pci,host=0003:31:00.0 -kernel /img/Image
-initrd /img/initramfs.cpio.gz
```

   以下qemu参数说明。

```
qemu-system-aarch64
```

   Rockchip buildroot编译出来的qemu二进制文件，放在host os根文件系统的bin/目录下。

```
-m 128
```

   为虚拟机创建128M的内存空间。

```
highmem=off
```

   使用低32位地址空间。

```
-device vfio-pci,host=0000:01:00.0
-device vfio-pci,host=0003:31:00.0
```

   使用PCIe设备地址作为参数创建vfio设备，用于虚拟机访问。

```
-kernel /img/Image -initrd /img/initramfs.cpio.gz
```

   分别是guest os内核固件，guest os根文件系统固件，这里将这两个固件放在host os根文件系统的img目录。

4. 待虚拟机启动完成，使用以下命令配置PCIe网卡

```
ifconfig eth0 up

ifconfig eth0 192.168.31.120 netmask 255.255.255.0

route add default gw 192.168.31.1
```

   地址以及网关根据实际情况配置。

5. 使用以下命令测试网络连接

```
ping 8.8.8.8
```

正常情况下，能看到ping信息。

6. 配置SR-IOV

   测试SR-IOV的功能时，lspci显示的设备地址为

```
00:00.0 Class 0604: Device 1d87:3588 (rev 01)
01:00.0 Class 0200: Device 8086:1521 (rev 01)
01:00.1 Class 0200: Device 8086:1521 (rev 01)
```

   这是一个具备两个网口的网卡设备，因此有两个PF，分别是`01:00.0`和`01:00.1`。

   - 创建VF，需要先从sriov_totalvfs获取最大允许创建的数量。

```
cat /sys/bus/pci/devices/0000:01:00.0/sriov_totalvfs
7
echo 2 > /sys/bus/pci/devices/0000:01:00.0/sriov_numvfs
```

     Host串口有如下打印表示创建VF成功

```
pci 0000:01:10.0: [8086:1520] type 00 class 0x020000 PCIe Endpoint
pci 0000:01:10.4: [8086:1520] type 00 class 0x020000 PCIe Endpoint
```

     以上log显示创建了两个VF分别是`0000:01:10.0`和`0000:01:10.4`。

     这时候再运行lspci，会显示多了两个设备。

```
00:00.0 Class 0604: Device 1d87:3588 (rev 01)
01:00.0 Class 0200: Device 8086:1521 (rev 01)
01:00.1 Class 0200: Device 8086:1521 (rev 01)
01:10.0 Class 0200: Device 8086:1520 (rev 01)
01:10.4 Class 0200: Device 8086:1520 (rev 01)
```

   - 挂载驱动

     以上只是创建了VF，还没挂载驱动，使用以下命令挂载驱动。

```
echo vfio-pci > /sys/bus/pci/devices/0000:01:10.0/driver_override
echo 0000:01:10.0 > /sys/bus/pci/drivers_probe
echo vfio-pci > /sys/bus/pci/devices/0000:01:10.4/driver_override
echo 0000:01:10.4 > /sys/bus/pci/drivers_probe
```

   - 详细设备状态

```
lspci -v
00:00.0 Class 0604: Device 1d87:3588 (rev 01)
        Flags: bus master, fast devsel, latency 0, IRQ 124, IOMMU group
14
        Bus: primary=00, secondary=01, subordinate=ff, sec-latency=0
        I/O behind bridge: 1000-1fff [size=4K] [16-bit]
        Memory behind bridge: f0200000-f04fffff [size=3M] [32-bit]
        Prefetchable memory behind bridge: 900000000-9000fffff [size=1M]
[32-bit]
        Expansion ROM at f0500000 [virtual] [disabled] [size=64K]
        Capabilities: [40] Power Management version 3
        Capabilities: [50] MSI: Enable+ Count=16/32 Maskable+ 64bit+
```

```
        Capabilities: [70] Express Root Port (Slot-), IntMsgNum 8
        Capabilities: [b0] MSI-X: Enable- Count=128 Masked-
        Capabilities: [100] Advanced Error Reporting
        Capabilities: [148] Secondary PCI Express
        Capabilities: [190] L1 PM Substates
        Capabilities: [1d0] Vendor Specific Information: ID=0002 Rev=4
Len=100 <?>
     Capabilities: [2d0] Vendor Specific Information: ID=0006 Rev=0
Len=018 <?>
        Kernel driver in use: pcieport

01:00.0 Class 0200: Device 8086:1521 (rev 01)
        Subsystem: Device 8086:0002
        Flags: bus master, fast devsel, latency 0, IRQ 123, IOMMU group
15
        Memory at f0200000 (32-bit, non-prefetchable) [size=512K]
        I/O ports at 1000 [disabled] [size=32]
        Memory at f0400000 (32-bit, non-prefetchable) [size=16K]
        Expansion ROM at f0280000 [virtual] [disabled] [size=512K]
        Capabilities: [40] Power Management version 3
        Capabilities: [50] MSI: Enable- Count=1/1 Maskable+ 64bit+
        Capabilities: [70] MSI-X: Enable+ Count=10 Masked-
        Capabilities: [a0] Express Endpoint, IntMsgNum 0
        Capabilities: [100] Advanced Error Reporting
        Capabilities: [140] Device Serial Number 1c-86-0b-ff-ff-2d-d2-e8
        Capabilities: [150] Alternative Routing-ID Interpretation (ARI)
        Capabilities: [160] Single Root I/O Virtualization (SR-IOV)
        Capabilities: [1a0] Transaction Processing Hints
        Capabilities: [1c0] Latency Tolerance Reporting
     Capabilities: [1d0] Access Control Services
        Kernel driver in use: igb

01:00.1 Class 0200: Device 8086:1521 (rev 01)
        Subsystem: Device 8086:0002
        Flags: bus master, fast devsel, latency 0, IRQ 145, IOMMU group
16
        Memory at f0300000 (32-bit, non-prefetchable) [size=512K]
        I/O ports at 1020 [disabled] [size=32]
        Memory at f0404000 (32-bit, non-prefetchable) [size=16K]
        Expansion ROM at f0380000 [virtual] [disabled] [size=512K]
        Capabilities: [40] Power Management version 3
        Capabilities: [50] MSI: Enable- Count=1/1 Maskable+ 64bit+
        Capabilities: [70] MSI-X: Enable+ Count=10 Masked-
        Capabilities: [a0] Express Endpoint, IntMsgNum 0
        Capabilities: [100] Advanced Error Reporting
        Capabilities: [140] Device Serial Number 1c-86-0b-ff-ff-2d-d2-e8
        Capabilities: [150] Alternative Routing-ID Interpretation (ARI)
        Capabilities: [160] Single Root I/O Virtualization (SR-IOV)
        Capabilities: [1a0] Transaction Processing Hints
     Capabilities: [1d0] Access Control Services
        Kernel driver in use: igb

01:10.0 Class 0200: Device 8086:1520 (rev 01)
        Subsystem: Device 8086:0002
        Flags: fast devsel, IOMMU group 21
        Memory at 900000000 (64-bit, prefetchable) [virtual] [size=16K]
        Memory at 900020000 (64-bit, prefetchable) [virtual] [size=16K]
        Capabilities: [70] MSI-X: Enable- Count=3 Masked-
```

```
        Capabilities: [a0] Express Endpoint, IntMsgNum 0
        Capabilities: [100] Advanced Error Reporting
        Capabilities: [150] Alternative Routing-ID Interpretation (ARI)
        Capabilities: [1a0] Transaction Processing Hints
      Capabilities: [1d0] Access Control Services
        Kernel driver in use: vfio-pci

01:10.4 Class 0200: Device 8086:1520 (rev 01)
        Subsystem: Device 8086:0002
        Flags: fast devsel, IOMMU group 22
        Memory at 900004000 (64-bit, prefetchable) [virtual] [size=16K]
        Memory at 900024000 (64-bit, prefetchable) [virtual] [size=16K]
        Capabilities: [70] MSI-X: Enable- Count=3 Masked-
        Capabilities: [a0] Express Endpoint, IntMsgNum 0
        Capabilities: [100] Advanced Error Reporting
        Capabilities: [150] Alternative Routing-ID Interpretation (ARI)
        Capabilities: [1a0] Transaction Processing Hints
        Capabilities: [1d0] Access Control Services
      Kernel driver in use: vfio-pci
```

可以看出`01:10.0`和`01:10.4`两个设备挂载了vfio-pci驱动，并且分配了各自的BAR空间。

7. 运行qemu测试SR-IOV

```
qemu-system-aarch64 -L /img -no-reboot -nographic -monitor none -m 256 -
machine virt,highmem=off -net none -device vfio-pci,host=0000:01:10.0 -
device vfio-pci,host=0000:01:10.4 -cpu host -enable-kvm -kernel /img/Image -
initrd /img/initramfs_guest.cpio.gz
```

同第3步的区别是设备地址替换成VF地址。

8. 通过VF启动的虚拟机，lspci和ifconfig能得到如下信息：

```
lspci -v
00:00.0 Class 0600: Device 1b36:0008
        Subsystem: Device 1af4:1100
        Flags: fast devsel

00:01.0 Class 0200: Device 8086:1520 (rev 01)
        Subsystem: Device 8086:0002
        Flags: bus master, fast devsel, latency 0
        Memory at 10000000 (64-bit, prefetchable) [size=16K]
        Memory at 10004000 (64-bit, prefetchable) [size=16K]
        Capabilities: [70] MSI-X: Enable+ Count=3 Masked-
        Capabilities: [a0] Express Root Complex Integrated Endpoint,
IntMsgNum 0
        Capabilities: [100] Advanced Error Reporting
        Capabilities: [1a0] Transaction Processing Hints
        Capabilities: [1d0] Access Control Services
        Kernel driver in use: igbvf

00:02.0 Class 0200: Device 8086:1520 (rev 01)
        Subsystem: Device 8086:0002
        Flags: bus master, fast devsel, latency 0
        Memory at 10008000 (64-bit, prefetchable) [size=16K]
        Memory at 1000c000 (64-bit, prefetchable) [size=16K]
        Capabilities: [70] MSI-X: Enable+ Count=3 Masked-
```

```
        Capabilities: [a0] Express Root Complex Integrated Endpoint,
IntMsgNum 0
        Capabilities: [100] Advanced Error Reporting
        Capabilities: [1a0] Transaction Processing Hints
        Capabilities: [1d0] Access Control Services
        Kernel driver in use: igbvf

ifconfig -a
eth0      Link encap:Ethernet  HWaddr 06:7B:06:99:65:04
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

eth1      Link encap:Ethernet  HWaddr 82:00:B8:2E:F4:64
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          LOOPBACK  MTU:65536  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

有两个网卡设备，这两个网卡设备由VF模拟，Guest内核驱动网卡工作的驱动代码是igbvf，可以看出，虚拟机的VF网卡要能正常工作，虚拟机内核需要有相应的VF驱动。

用以下命令分别配置网卡，均能ping通，说明VF工作正常

```
ifconfig eth0 up
ifconfig eth0 192.168.31.121 netmask 255.255.255.0
route add default gw 192.168.31.1

ifconfig eth1 up
ifconfig eth1 192.168.31.122 netmask 255.255.255.0
route add default gw 192.168.31.1
```

需要说明的是当前测试环境暂无法启动一个以上的虚拟机，因此在一个虚拟机里同时只能测试一个网卡，另外，Host和Guest的网卡也能同时工作。

```
64 bytes from 8.8.8.8: seq=141 ttl=108 time=189.385 ms
64 bytes from 192.168.31.1: seq=103 ttl=64 time=0.655 ms
64 bytes from 8.8.8.8: seq=142 ttl=108 time=189.099 ms
64 bytes from 192.168.31.1: seq=104 ttl=64 time=0.597 ms
64 bytes from 8.8.8.8: seq=143 ttl=108 time=189.428 ms
64 bytes from 192.168.31.1: seq=105 ttl=64 time=0.584 ms
64 bytes from 8.8.8.8: seq=144 ttl=108 time=189.002 ms
64 bytes from 192.168.31.1: seq=106 ttl=64 time=0.547 ms
64 bytes from 8.8.8.8: seq=145 ttl=108 time=189.032 ms
64 bytes from 192.168.31.1: seq=107 ttl=64 time=0.532 ms
64 bytes from 8.8.8.8: seq=146 ttl=108 time=189.673 ms
64 bytes from 192.168.31.1: seq=108 ttl=64 time=0.630 ms
```

```
64 bytes from 8.8.8.8: seq=147 ttl=108 time=189.086 ms
64 bytes from 192.168.31.1: seq=109 ttl=64 time=0.611 ms
64 bytes from 8.8.8.8: seq=148 ttl=108 time=189.729 ms
64 bytes from 192.168.31.1: seq=110 ttl=64 time=0.650 ms
64 bytes from 8.8.8.8: seq=149 ttl=108 time=189.450 ms
```

`8.8.8.8`和`192.168.31.1`分别来自Guest和Host的Ping通信息。

9. 使用以下命令测试NVME SSD

```
mount /dev/nvme0n1 /tmp/
dd if=/dev/random of=/tmp/test bs=1M count=100
```

正常情况下，能看到如下信息：

```
100+0 records in
100+0 records out
104857600 bytes (100.0MB) copied, 0.446296 seconds, 224.1MB/s
```

4. 遗留问题

1. 在虚拟机里面PCIe设备默认被设置为coherent属性，所以申请的dma buffer被设置成cacheable，由于RK3588不具备cache一致性的功能，从而导致数据一致性的问题，需要在虚拟机PCIe设备probe时候，手动设置为non-coherent，如下，才能规避数据一致性问题

```
dev->archdata.dma_coherent = false;
```

以develop-6.12 NVME SSD和R8169网卡驱动为例：

```
git diff
diff --git a/drivers/net/ethernet/realtek/r8169_main.c
b/drivers/net/ethernet/realtek/r8169_main.c
index 9c601f271c02..25e443d28294 100644
--- a/drivers/net/ethernet/realtek/r8169_main.c
+++ b/drivers/net/ethernet/realtek/r8169_main.c
@@ -5385,6 +5385,8 @@ static int rtl_init_one(struct pci_dev *pdev, const
struct pci_device_id *ent)
        u32 txconfig;
        u16 xid;

+       pdev->dev.dma_coherent = false;
+
        dev = devm_alloc_etherdev(&pdev->dev, sizeof (*tp));
        if (!dev)
                return -ENOMEM;
diff --git a/drivers/nvme/host/pci.c b/drivers/nvme/host/pci.c
index 2c6d9506b172..855f725e87aa 100644
--- a/drivers/nvme/host/pci.c
+++ b/drivers/nvme/host/pci.c
@@ -3417,6 +3417,8 @@ static int nvme_probe(struct pci_dev *pdev, const
struct pci_device_id *id)
        struct nvme_dev *dev;
```

```
        int result = -ENOMEM;

+       pdev->dev.dma_coherent = false;
+
        dev = nvme_pci_alloc_dev(pdev, id);
        if (IS_ERR(dev))
                return PTR_ERR(dev);
```

需要说明的是新内核比如develop-6.1之后，dma_coherent成员挪到struct device结构体里。

2. 需要修改drivers/iommu/arm/arm-smmu-v3/arm-smmu-v3.c，否则qemu运行时候会出现如下错误：

```
vfio-pci 0003:31:00.0: probe with driver vfio-pci failed with error -22
```

出现这个问题的原因是，VFIO需要设备的IOMMU支持CACHE一致性，RK3588的SMMU还不支持CACHE一致性，因此，需要对arm-smmu-v3.c做一下修改：

```
git diff drivers/iommu/arm/arm-smmu-v3/arm-smmu-v3.c
diff --git a/drivers/iommu/arm/arm-smmu-v3/arm-smmu-v3.c
b/drivers/iommu/arm/arm-smmu-v3/arm-smmu-v3.c
index 6bf8ecbbe0c2..9c3ea0b842e5 100644
--- a/drivers/iommu/arm/arm-smmu-v3/arm-smmu-v3.c
+++ b/drivers/iommu/arm/arm-smmu-v3/arm-smmu-v3.c
@@ -2291,6 +2291,7 @@ static bool arm_smmu_capable(struct device *dev, enum
iommu_cap cap)

        switch (cap) {
        case IOMMU_CAP_CACHE_COHERENCY:
+               return true;
                /* Assume that a coherent TCU implies coherent TBUs */
                return master->smmu->features & ARM_SMMU_FEAT_COHERENCY;
        case IOMMU_CAP_NOEXEC:
```

因此，未来SMMU的feature必须考虑包含CACHE一致性。

3. 需要修改drivers/iommu/iommu.c，否则VF和PF会被分配在同一个IOMMU GROUP内，这导致所有的VF/PF都只能直通给同一个虚拟机，因此需要打以下补丁强制为每个VF/PF分配独立的IOMMU GROUP。

```
--- a/drivers/iommu/iommu.c
+++ b/drivers/iommu/iommu.c
@@ -1548,7 +1548,7 @@ struct iommu_group *pci_device_group(struct device
*dev)
                if (!bus->self)
                        continue;

-               if (pci_acs_path_enabled(bus->self, NULL, REQ_ACS_FLAGS))
+               if (!pci_acs_path_enabled(bus->self, NULL, REQ_ACS_FLAGS))
                        break;

                pdev = bus->self;
```

未来可以继续验证搭配具备ACS功能的Switch是否能解决VF/PF同组的问题。

4. 和其他设备一样，VF驱动也要设置设备coherent属性为False

```
--- a/drivers/net/ethernet/intel/igbvf/netdev.c
+++ b/drivers/net/ethernet/intel/igbvf/netdev.c
@@ -2706,6 +2706,8 @@ static int igbvf_probe(struct pci_dev *pdev, const
struct pci_device_id *ent)
        const struct igbvf_info *ei = igbvf_info_tbl[ent->driver_data];
        int err;

+       pdev->dev.dma_coherent = false;
+
        err = pci_enable_device_mem(pdev);
        if (err)
                return err;
```

5. 修改dtb

在Guest环境中，PCIe设备的 `pdev->dev.dma_coherent` 属性为 true，其根本原因是 QEMU 生成的 FDT（设备树）中，PCIe 控制器节点被添加了 `dma-coherent` 属性。由于 PCIe 设备会继承 PCIe 控制器的 `dma_coherent` 属性，因此导致 `pdev->dev.dma_coherent` 为 true。为了解决这一问题，无需在每个 PCIe 驱动中单独修改 `pdev->dev.dma_coherent = false`，只需将 PCIe 控制器的属性强制设置为 `dma-noncoherent` 即可。具体方法如下：

1. 生成dtb

   ```
   qemu-system-aarch64 -L /img -no-reboot -nographic -monitor none -m 256 -
   machine virt,highmem=off,dumpdtb=virt64.dtb -net none  -cpu host -enable-kvm
   ```

   以上命令可以在Host生成virt64.dtb。

2. 修改dtb

   ```
   dtc -I dtb -O dts virt64.dtb > virt64.dts
   ```

   在PCIe控制器节点将dma-coherent修改为dma-noncoherent;

3. 重新生成dtb

   ```
   dtc -I dts -O dtb virt64.dts -o virt64.dtb
   ```

4. 运行qemu时指定dtb

   ```
   qemu-system-aarch64 -L /img -no-reboot -nographic -monitor none -m 256 -
   machine virt,highmem=off -net none -device vfio-pci,host=0000:01:10.0 -
   device vfio-pci,host=0000:01:10.4 -cpu host -enable-kvm -kernel /img/Image -
   initrd /img/initramfs_guest.cpio.gz -dtb virt64.dtb
   ```

经过以上措施，Guest的PCIe设备在枚举的时候，`pdev->dev.dma_coherent` 被默认设置为false。

这个做法的好处是不用修改具体的Guest内核的PCIe设备驱动。